

Short Report: AI Tools and Workflow

Course: CCS3600 Artificial Intelligence (Natural Language Processing) Project: AI-Assisted Teaching Content Enhancement, Topic 10: Back Propagation This is the tools-and-workflow report (Deliverable 4). Its companion, the prompt examples, is a separate document (Deliverable 5).

This report covers the **whole** project, not just the slides. The brief asks for three things — improved slides, a lecture video, and an auto-graded activity — plus the report and prompt examples. I did all of those, and then I went one step further and pulled every piece into a single trilingual website that is the actual thing handed in. So the workflow below runs from the original PowerPoint all the way to a deployed site.

What I started with and what I ended up with

I started with a 32-slide deck on Back Propagation Neural Networks that was very text-heavy — most pages were just paragraphs pasted onto a slide — and the project brief.

What I ended up with, in order of the brief's deliverables:

1. **Improved slides** — a 22-slide deck that is still a real, editable PowerPoint file (not images pasted into slides), with four custom illustrations.
2. **Lecture video** — a roughly 17 to 18 minute narrated video with a subtitle file that lines up with the audio word for word.
3. **Auto-graded quiz** — 20 questions (15 multiple-choice, 5 true/false), every one with a single unambiguous answer and a written explanation, grounded in the original courseware.
4. **This report.**
5. **The prompt examples** (separate document).

And on top of those five, one extra thing the brief did not ask for: a **trilingual website** (English, 简体中文, Bahasa Melayu) that presents all of the above in one place, deployed live so it can be opened in a browser instead of downloaded as loose files.

I got from 32 slides to 22 by merging repeated content, not by cutting topics. Everything from the original is still there: the chain-rule derivation, the BP versus BPTT distinction, the history, and the applications. The quiz is built from that same content so it tests the lecture, not trivia.

Tools I used

Tool	What it did	Why this one
Claude Code (Claude Opus 4.7, 1M context)	The editor and orchestrator across every stage — content analysis, design, SVG authoring, narration, quiz drafting, the website build, and the deployment steps. Also ran sub-agents for planning (see below).	Long-context reasoning and precise coordinate/layout control; it kept 22 hand-placed vector slides consistent where smaller models drifted.
PPT Master (open source)	Turned the design into a <i>natively editable</i> PPTX via an AI-writes-SVG → script-converts-to-DrawingML pipeline.	Real PowerPoint objects out the other side — the reason every text box and shape stays selectable and editable. Explained further down.
OpenAI gpt-image-2	The four illustrations (cover, applications divider, feed-forward diagram, backprop flow diagram).	Stayed on-style across a set once the prompt pinned the exact colours and projection.
Microsoft Edge neural TTS (edge-tts, en-US-AvaMultilingualNeural)	The narration and the word-level subtitle timing.	Natural enough for a lecture, and it returns real word-level timestamps so subtitles match the voice instead of being estimated.
Google Gemini image API	Configured as a fallback if gpt-image-2 went down.	Redundancy only — it was never needed and nothing in the deck came from it.
ffmpeg 8.1	Re-encoded the lecture video so it could ship inside the website.	Two-pass H.264 control over bitrate and size; nothing uploaded to a third party.
Next.js 16 (App Router) + next-intl + Tailwind v4 + KaTeX + react-markdown	The website itself — routing, the three languages, the formula rendering, the quiz, the document pages.	First-party Vercel target and a stateful, client-side quiz that grades itself.
git + GitHub CLI (gh)	Version control and the private repository.	Standard, and a private repo protects the worked solution and answer key.

Tool	What it did	Why this one
Vercel	Hosting the live site.	First-party Next.js hosting, free tier, deploy straight from the repo.

These map onto the assignment's "Suggested AI Tools" categories: a ChatGPT/Gemini-class generative model (used here via Claude plus gpt-image-2) for the slides, an AI voice + subtitle pipeline for the video, and AI-assisted authoring for the quiz — with everything reviewed by hand.

How the work actually went

Roughly in order.

Stage 1 — Slides. First I converted the original PPTX and the project brief into Markdown, so the content and the marking criteria were both in front of me while I worked. Then I spent time on structure before touching any visuals: which slides should merge, what the information hierarchy on each page should be, and I locked a design spec early — a dark "neural network" look with a fixed four-colour palette (deep slate #0F172A, blue #3B82F6, cyan #22D3EE, purple #8B5CF6), a type scale, and a per-slide plan for icons and images. Locking that early is what kept 22 separately generated slides looking like one deck. The four images were generated next, against detailed prompts that fixed the palette and style. Slides were then built one page at a time as SVG following the spec. After that I ran an automated check over every slide for the obvious failure modes — wrong canvas size, broken image or icon references, anything malformed — which caught real problems (four broken image paths and several missing icon references) that I fixed before moving on. Finally the SVGs were exported to a native PPTX.

Stage 2 — Lecture video. I wrote a speaking script for all 22 slides and ran a timing test. The first version ran long. Rather than speed the voice up — which sounds rushed and is an obvious giveaway — I cut the script down until it landed inside the 15 to 20 minute window at a normal pace. The per-slide audio was then generated with the word-timed subtitles in the same pass, slide auto-advance timings were set from each slide's real audio length, and the SRT cues were stitched together with the right offsets. The first subtitle track had been timed by estimation and drifted ahead of the voice; I threw it out and regenerated it from real word boundaries.

Stage 3 — Auto-graded quiz. The brief wants at least ten questions that grade automatically, with an answer scheme or auto-feedback. I built 20: fifteen multiple-choice and five true/false, one mark each. Every question is grounded in the original courseware

and its explanation notes which slide it comes from, so the quiz tests the lecture rather than outside trivia. A few questions are deliberately built around the subtle point that backpropagation *computes* the gradient but does not *define how it is used* — that is the optimizer's job — because that distinction is the one students most often get wrong, and getting it into the quiz means I had to understand it myself, not just generate it. Every item has a single unambiguous key and a written explanation that doubles as the feedback shown after submission. The Markdown version is platform-neutral (it imports cleanly into Google Forms, Quizizz, or Kahoot), and on the website the same 20 questions run as a self-grading activity with instant per-question feedback and a live score.

Stage 4 — One trilingual website. Instead of handing in five loose files, I rebuilt everything as one site so a grader can open a URL and see the slides, watch the video, take the quiz, and read the report and prompts without downloading anything. The 22-slide deck is rebuilt as native, responsive sections with the formulas typeset properly in KaTeX (not screenshots). The whole thing is in three languages — English as the source of truth, then Simplified Chinese, then Bahasa Melayu — with any string that is not yet finalised in a language falling back to English so a late translation can never break the site. The quiz is a client-side component, so grading happens in the browser with no backend to stand up.

Stage 5 — Re-encode, repository, deploy. The lecture video master was about 207 MB, too big to commit. I rejected Git LFS because Vercel does not fetch LFS objects by default, which would have left a broken video in production. Instead I re-encoded it with ffmpeg — two-pass H.264, roughly 600 kbps video plus 96 kbps AAC audio, keeping 1080p — down under GitHub's 100 MB per-file hard limit, and I left the video source pluggable so it can be swapped for a YouTube embed by changing one config value if it ever needs to be. The code went into a **private** GitHub repository (this protects the worked solution and the answer key — graders use the live URL, not the source), with the assignment brief and the raw original decks deliberately excluded for academic-integrity reasons. Finally the repo was deployed on Vercel as a first-party Next.js app.

Before the build I had the agent run a short planning pass with sub-agents — a few read-only exploration agents to map the existing material and a couple of planning agents to choose the stack and the i18n approach — so the structural decisions (Next.js over a static-site generator, quiz as a client component, phased translations) were made deliberately rather than discovered halfway through, given the deadline.

What I had to fix and verify myself

This was not a generate-once-and-submit job. The brief is explicit that AI output must be reviewed and that copying unmodified generation is not acceptable, so:

- The 32 → 22 restructuring was my editorial call, not the model's. I decided what merged.
- I checked the technical content on every slide against the original lecture: the formulas, the chain-rule explanation, the BP/BPTT distinction, the timeline.
- The broken image paths and wrong icon references the quality check flagged were fixed manually.
- The narration was too long on the first pass, so I rewrote it shorter instead of speeding up playback.
- The first subtitle track drifted; I regenerated it from real word boundaries.
- Every quiz question and its key was checked against the source courseware. The auto-grader is only trustworthy if the keys are right, so I verified all twenty and the slide each maps to, rather than trusting the generated answers.
- I made the deployment decisions myself and for stated reasons: no Git LFS (it breaks on Vercel), re-encode in-repo with a YouTube fallback kept available, private repo to protect the solution, brief and raw decks kept out of version control.

Why PPT Master and not a one-click tool

I deliberately did not use a browser "type a topic, get slides" generator for the final deck.

The output is a real PowerPoint. Most AI slide tools either embed each slide as a flat image or render HTML that falls apart on PPTX export. PPT Master goes AI → SVG → DrawingML, so the exported deck is made of normal PowerPoint objects. That matters here directly, because the brief asks for professional, modifiable slides and explicitly says not to hand in unmodified AI output. With this approach I could actually edit the AI's work, and I did.

It is free and open source, so the only cost was my own model usage. It runs locally, so the source file, the generation, and the export all happen on my machine — nothing goes to a third-party slide service. The honest downside is that it needs local setup and is slow, because it generates pages one at a time on purpose to keep the deck consistent. For a course project where editability and being able to verify every slide mattered more than speed, that trade was worth it, and it is the safer position on the academic-integrity rule.

The same reasoning runs through the rest of the stack: ffmpeg over an online compressor (the video never leaves the machine), a private repo over a public one (the

solution and answer key stay protected), and a real Next.js app over a slide-export SaaS (the quiz can actually grade itself, and the whole package lives at one URL).

What this produced

Brief deliverable	What was produced
1 — Improved slide presentation	A native, editable 22-slide PPTX (also rebuilt as responsive web sections; PPTX and PDF downloadable from the site)
2 — AI-generated lecture video	The narrated lecture, about 17–18 minutes, with a word-accurate SRT subtitle track, played in-page on the site
3 — Auto-gradable quiz	20 questions (15 MC + 5 T/F), single unambiguous key per item, written explanations as auto-feedback; runs self-grading on the site and imports cleanly into Google Forms / Quizizz / Kahoot
4 — Short report	This document
5 — Prompt examples	The companion document
(Extra) Presentation form	One trilingual (EN / 中文 / Bahasa Melayu) website unifying all five, deployed on Vercel